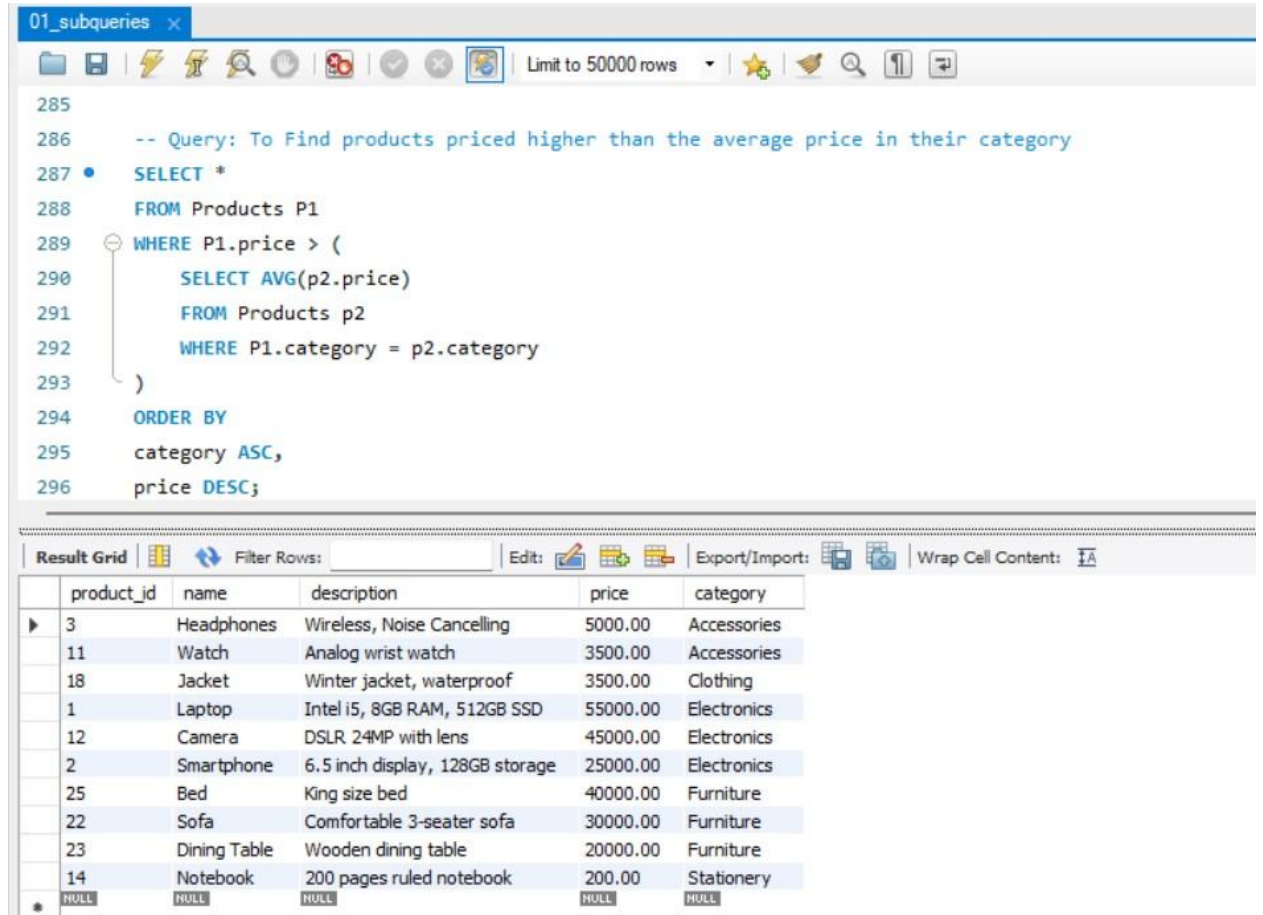


Name : More Shrinivas Balaji

Topic : Intermediate SQL Assignment 1

a) Find products priced higher than the average price in their category. Order By category ascending and price descending using Subquery.

- used correlated subquery so average is calculated per category
- compared each product price with its category average
- sorted results to see category-wise and highest prices first



The screenshot shows a SQL IDE window titled "01_subqueries". The query editor contains the following SQL code:

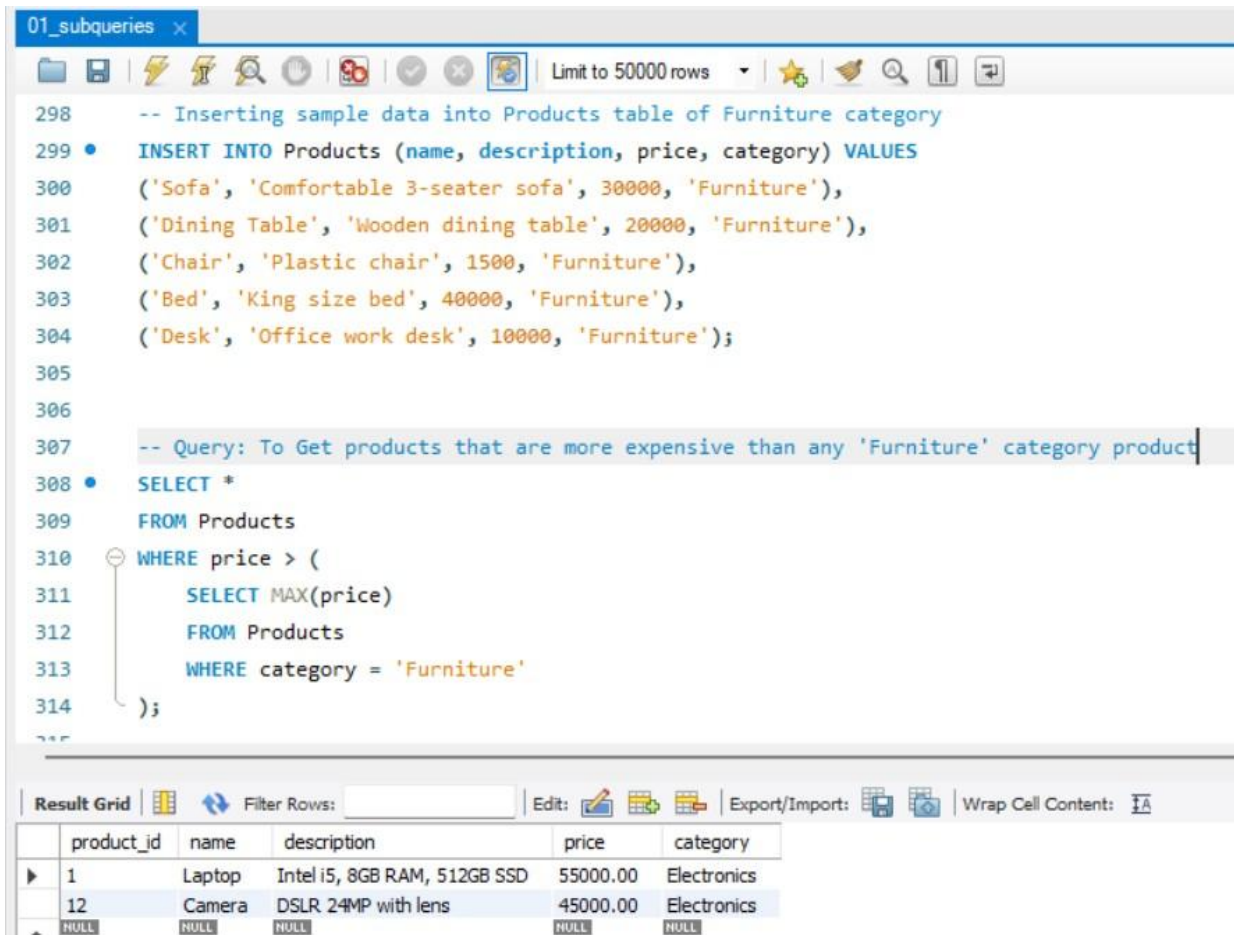
```
285
286 -- Query: To Find products priced higher than the average price in their category
287 • SELECT *
288 FROM Products P1
289 WHERE P1.price > (
290     SELECT AVG(p2.price)
291     FROM Products p2
292     WHERE P1.category = p2.category
293 )
294 ORDER BY
295     category ASC,
296     price DESC;
```

The results pane below the query editor displays a table with 6 columns: product_id, name, description, price, and category. The table contains 14 rows of data, sorted by category in ascending order and then by price in descending order. The last row shows a product with NULL values for price and category.

product_id	name	description	price	category
3	Headphones	Wireless, Noise Cancelling	5000.00	Accessories
11	Watch	Analog wrist watch	3500.00	Accessories
18	Jacket	Winter jacket, waterproof	3500.00	Clothing
1	Laptop	Intel i5, 8GB RAM, 512GB SSD	55000.00	Electronics
12	Camera	DSLR 24MP with lens	45000.00	Electronics
2	Smartphone	6.5 inch display, 128GB storage	25000.00	Electronics
25	Bed	King size bed	40000.00	Furniture
22	Sofa	Comfortable 3-seater sofa	30000.00	Furniture
23	Dining Table	Wooden dining table	20000.00	Furniture
14	Notebook	200 pages ruled notebook	200.00	Stationery
•	NULL	NULL	NULL	NULL

b) Get products that are more expensive than any 'Furniture' category product using Subquery

- first approach uses MAX to get highest Furniture price and compares with it



The screenshot shows a database IDE window titled "01_subqueries". The SQL editor contains the following code:

```
298 -- Inserting sample data into Products table of Furniture category
299 • INSERT INTO Products (name, description, price, category) VALUES
300 ('Sofa', 'Comfortable 3-seater sofa', 30000, 'Furniture'),
301 ('Dining Table', 'Wooden dining table', 20000, 'Furniture'),
302 ('Chair', 'Plastic chair', 1500, 'Furniture'),
303 ('Bed', 'King size bed', 40000, 'Furniture'),
304 ('Desk', 'Office work desk', 10000, 'Furniture');
305
306
307 -- Query: To Get products that are more expensive than any 'Furniture' category product
308 • SELECT *
309 FROM Products
310 WHERE price > (
311     SELECT MAX(price)
312     FROM Products
313     WHERE category = 'Furniture'
314 );
```

The result grid at the bottom shows the following data:

	product_id	name	description	price	category
▶	1	Laptop	Intel i5, 8GB RAM, 512GB SSD	55000.00	Electronics
	12	Camera	DSLR 24MP with lens	45000.00	Electronics
★	NULL	NULL	NULL	NULL	NULL

- second approach uses ALL to compare with all values directly

The screenshot shows a database IDE with a SQL editor and a result grid. The SQL editor contains the following code:

```
315
316  -- alternative approach with ALL keyword
317  • SELECT *
318    FROM Products
319  WHERE price > ALL(
320      SELECT (price)
321      FROM Products
322      WHERE category = 'Furniture'
323  );
324
325
```

The result grid below the query shows the following data:

	product_id	name	description	price	category
▶	1	Laptop	Intel i5, 8GB RAM, 512GB SSD	55000.00	Electronics
	12	Camera	DSLR 24MP with lens	45000.00	Electronics
*	NULL	NULL	NULL	NULL	NULL

c) Create a stored procedure to apply a discount on a specific category of products.

- created a procedure to apply discount category-wise so I don't have to repeat update queries

```
02_procedures* x
-- Procedure: UpdateProductDiscount
DELIMITER $$
326
327
328
329 CREATE PROCEDURE UpdateProductDiscount(
330     IN inputcategory VARCHAR(50),
331     IN inputdiscount INT
332 )
333 BEGIN
334     UPDATE Products
335     SET price = price - price * (inputdiscount / 100.0)
336     WHERE category = inputcategory;
337     -- This shows how many rows were updated
338     select ROW_COUNT() AS UpdatedRows;
339 END $$
340
341 DELIMITER ;
342
343 -- Before applying discount
344 SELECT * FROM Products WHERE category = 'Furniture';
```

- checking original prices of Furniture products before applying discount

Result Grid					
Filter Rows:		Edit:		Export/Import:	
				Wrap Cell Content:	
product_id	name	description	price	category	
8	Chair	Ergonomic office chair	7000.00	Furniture	
9	Table	Wooden study table	6000.00	Furniture	
20	Desk Lamp	LED desk lamp	1200.00	Furniture	
21	Sofa	Comfortable 3-seater sofa	30000.00	Furniture	
22	Dining Table	Wooden dining table	20000.00	Furniture	
23	Chair	Plastic chair	1500.00	Furniture	
24	Bed	King size bed	40000.00	Furniture	
25	Desk	Office work desk	10000.00	Furniture	
* NULL	NULL	NULL	NULL	NULL	

- calling the procedure with 10% discount for Furniture category

```

346 • CALL UpdateProductDiscount('Furniture', 10);
347 -- After applying discount
348 • SELECT * FROM Products WHERE category = 'Furniture';

```

Result Grid | Filter Rows: | Exports: | Wrap Cell Contents: |

UpdatedRows
8

- checking updated prices to confirm discount is applied correctly

```

347 -- After applying discount
348 • SELECT * FROM Products WHERE category = 'Furniture';
349

```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Contents: |

	product_id	name	description	price	category
▶	8	Chair	Ergonomic office chair	6300.00	Furniture
	9	Table	Wooden study table	5400.00	Furniture
	20	Desk Lamp	LED desk lamp	1080.00	Furniture
	21	Sofa	Comfortable 3-seater sofa	27000.00	Furniture
	22	Dining Table	Wooden dining table	18000.00	Furniture
	23	Chair	Plastic chair	1350.00	Furniture
	24	Bed	King size bed	36000.00	Furniture
	25	Desk	Office work desk	9000.00	Furniture
•	total	total	total	total	total

d) Create a trigger to update a customer's total spending whenever they place a new order.

- added a column to store total spending of each customer
- calculating total spending from past orders and updating existing records

```

350 -- Trigger to update a customer's total spending
351 -- I added this column because we need a place to store the total spending of each customer
352 • ALTER TABLE Customers
353 ADD total_spending DECIMAL(10,2) DEFAULT 0;
354
355 -- I am calculating the sum of all past orders for each customer and storing it in total_spending
356 • UPDATE Customers C
357 SET total_spending = (
358 SELECT COALESCE(SUM(O.total_amount), 0)
359 FROM Orders O
360 WHERE C.customer_id = O.customer_id
361 );
362

```


- created a trigger to automatically update total spending when a new order is inserted
- checking current total spending before inserting new order

The screenshot shows a SQL IDE window titled "04_null_handling* x". The SQL editor contains the following code:

```

366 DELIMITER $$
367 • CREATE TRIGGER afterspent
368 AFTER INSERT ON Orders
369 FOR EACH ROW
370 BEGIN
371     UPDATE Customers C
372     SET C.total_spending = C.total_spending + NEW.total_amount
373     WHERE C.customer_id = NEW.customer_id;
374 END $$
375 DELIMITER ;
376
377 • -- Before inserting new order
378 SELECT customer_id, total_spending FROM Customers WHERE customer_id = 1;
379

```

Below the editor is a "Result Grid" showing the results of the query in line 378:

customer_id	total_spending
1	235000.00

- inserting a new order to check whether trigger updates total spending correctly
- verifying total spending is updated automatically after insert

The screenshot shows the same SQL IDE window. The SQL editor now includes an insert statement:

```

366 DELIMITER $$
367 • CREATE TRIGGER afterspent
368 AFTER INSERT ON Orders
369 FOR EACH ROW
370 BEGIN
371     UPDATE Customers C
372     SET C.total_spending = C.total_spending + NEW.total_amount
373     WHERE C.customer_id = NEW.customer_id;
374 END $$
375 DELIMITER ;
376
377 • -- Before inserting new order
378 SELECT customer_id, total_spending FROM Customers WHERE customer_id = 1;
379
380 • INSERT INTO Orders (customer_id, product_id, quantity, order_date, status, payment_method, total_amount)
381 VALUES (1, 1, 1, '2026-03-01', 'Pending', 'UPI', 55000.00);
382
383 -- After inserting new order
384 • SELECT customer_id, total_spending FROM Customers WHERE customer_id = 1;
385

```

The "Result Grid" now shows the result of the query in line 384:

customer_id	total_spending
1	290000.00

e) Retrieve all customers along with their latest order details, showing "No orders placed" if no orders exist. Using COALESCE.

- used COALESCE to show 'No orders placed' instead of NULL
- used LEFT JOIN so no customer gets missed
- used subquery to pick only the most recent order with ORDER BY + LIMIT
- matched that order using order_id

The screenshot shows a SQL query editor window titled "D4_null_handling". The query is as follows:

```
388 -- Query : To Retrieve all customers along with their latest order details, showing "No orders placed" if no orders exist. Using COALESCE
389 • SELECT
390     c.customer_id,
391     c.name,
392     o.order_id,
393     o.product_id,
394     COALESCE(o.order_date, 'No orders placed') AS latest_order_date
395 FROM Customers c
396 LEFT JOIN Orders o
397     ON o.order_id = (
398     -- pick the order_id with the latest date for this customer
399     SELECT order_id
400     FROM Orders
401     WHERE customer_id = c.customer_id
402     ORDER BY order_date DESC, order_id ASC
403     LIMIT 1
404 )
```

Below the query editor, the "Result Grid" is displayed, showing the results of the query. The grid has columns: customer_id, name, order_id, product_id, and latest_order_date. The results are as follows:

customer_id	name	order_id	product_id	latest_order_date
1	Alice Johnson	1	1	2026-03-01
2	Bob Smith	2	2	2026-03-02
3	Charlie Brown	3	3	2026-03-03
4	David Lee	4	4	2026-03-04
5	Eva Green	5	5	2026-03-05
6	Frank White	6	6	2026-03-06
7	Grace Kim	7	7	2026-03-07
8	Hannah Scott	8	8	2026-03-08
9	Ian Clark	9	9	2026-03-09
10	Jane Doe	10	10	2026-03-10

- f) Find customers whose email and mobile number are the same. If they are the same, return NULL instead of duplicate values.
- used NULLIF to return NULL when email and mobile are same
 - otherwise it keeps the email value

The screenshot shows a SQL IDE window titled "04_null_handling*" and "SQL File 3*". The SQL editor contains the following code:

```
408 • INSERT INTO Customers (customer_id, name, email, mobile, age) VALUES
409     (23, 'Frank Miller', 'frank@gmail.com', 'frank@gmail.com', 29),
410     (24, 'Grace Hopper', 'grace@gmail.com', 'grace@gmail.com', 32),
411     (25, 'Hank Pym', 'hank@gmail.com', 'hank@gmail.com', 40),
412     (26, 'Ivy Chen', 'ivy@gmail.com', 'ivy@gmail.com', 27),
413     (27, 'Jack Black', 'jack@gmail.com', 'jack@gmail.com', 35);
414
415 • SELECT
416     customer_id,
417     name,
418     NULLIF(email, mobile) AS email
419 FROM Customers;
```

Below the SQL editor is the "Result Grid" showing the output of the query. It has columns for "customer_id", "name", and "email". The results are as follows:

customer_id	name	email
15	Oscar Wilde	oscar15@gmail.com
16	Paula Adams	paula16@gmail.com
17	Quincy Jones	quincy17@gmail.com
18	Rachel Moore	rachel18@gmail.com
19	Steve Rogers	steve19@gmail.com
20	Zara Lane	zara20@gmail.com
21	Test Student	teststudent@email....
23	Frank Miller	NULL
24	Grace Hopper	NULL
25	Hank Pym	NULL
26	Ivy Chen	NULL

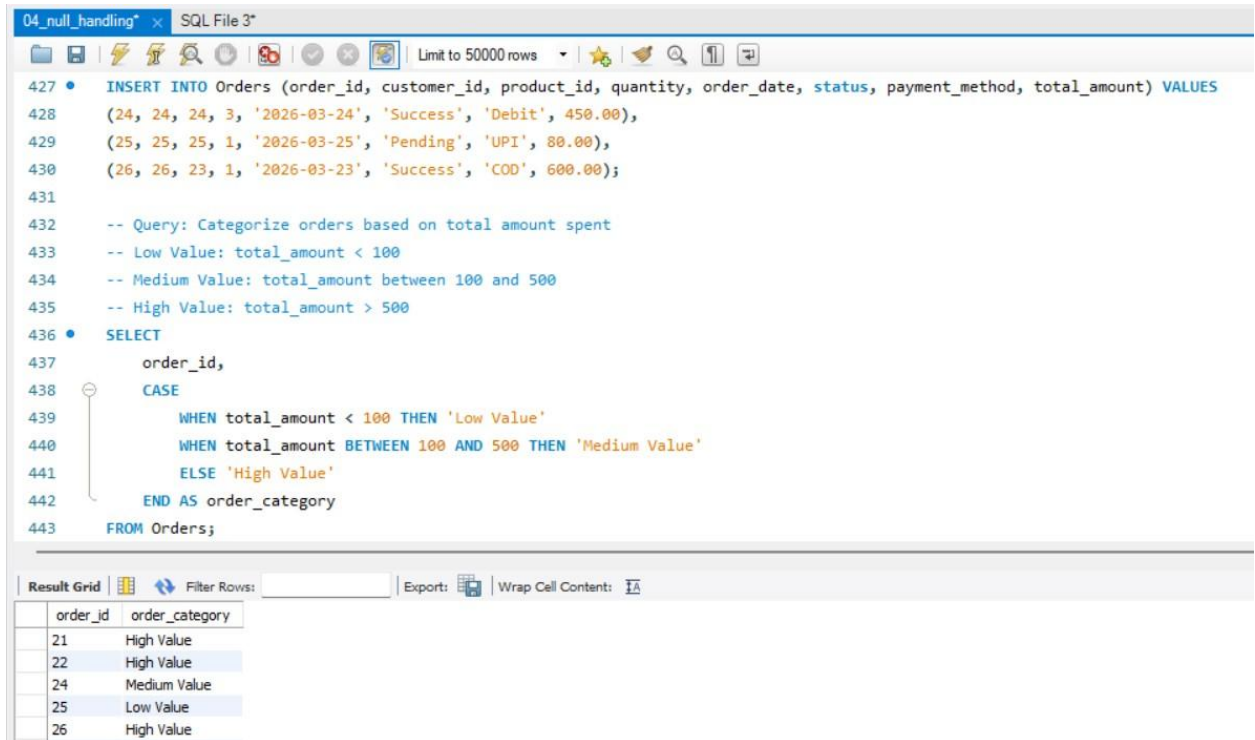
g) Categorize orders based on total amount spent:

Low Value: Less than \$100

Medium Value: \$100 - \$500

High Value: More than \$500

- added some test samples
- used CASE to assign labels based on total_amount range



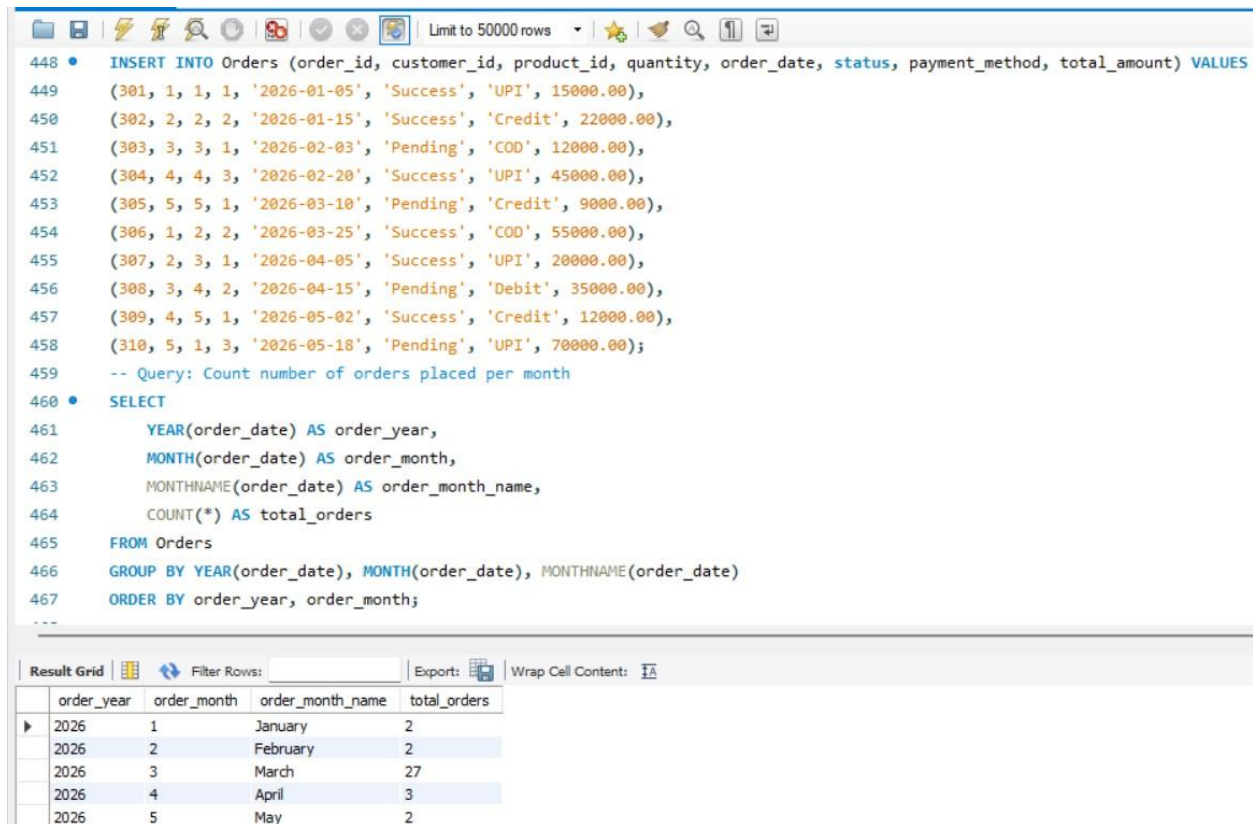
```
427 • INSERT INTO Orders (order_id, customer_id, product_id, quantity, order_date, status, payment_method, total_amount) VALUES
428 (24, 24, 24, 3, '2026-03-24', 'Success', 'Debit', 450.00),
429 (25, 25, 25, 1, '2026-03-25', 'Pending', 'UPI', 80.00),
430 (26, 26, 23, 1, '2026-03-23', 'Success', 'COD', 600.00);
431
432 -- Query: Categorize orders based on total amount spent
433 -- Low Value: total_amount < 100
434 -- Medium Value: total_amount between 100 and 500
435 -- High Value: total_amount > 500
436 • SELECT
437     order_id,
438     CASE
439         WHEN total_amount < 100 THEN 'Low Value'
440         WHEN total_amount BETWEEN 100 AND 500 THEN 'Medium Value'
441         ELSE 'High Value'
442     END AS order_category
443 FROM Orders;
```

Result Grid

order_id	order_category
21	High Value
22	High Value
24	Medium Value
25	Low Value
26	High Value

h) Find the number of orders placed per month.

- added sample data
- used YEAR, MONTH and MONTHNAME to group data
- counted orders per month to see trends over time



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, search, and execution. The main editor displays SQL code with line numbers 448 to 467. The code consists of an INSERT statement (lines 448-458) and a SELECT query (lines 460-467). The SELECT query uses YEAR, MONTH, and MONTHNAME functions to group data by year, month, and month name, and counts the total orders. Below the editor, the 'Result Grid' tab is active, showing the results of the SELECT query in a table format. The table has four columns: order_year, order_month, order_month_name, and total_orders. The results show data for the years 2026, with months 1 through 5 (January through May) and their corresponding total orders.

```
448 • INSERT INTO Orders (order_id, customer_id, product_id, quantity, order_date, status, payment_method, total_amount) VALUES
449     (301, 1, 1, 1, '2026-01-05', 'Success', 'UPI', 15000.00),
450     (302, 2, 2, 2, '2026-01-15', 'Success', 'Credit', 22000.00),
451     (303, 3, 3, 1, '2026-02-03', 'Pending', 'COD', 12000.00),
452     (304, 4, 4, 3, '2026-02-20', 'Success', 'UPI', 45000.00),
453     (305, 5, 5, 1, '2026-03-10', 'Pending', 'Credit', 9000.00),
454     (306, 1, 2, 2, '2026-03-25', 'Success', 'COD', 55000.00),
455     (307, 2, 3, 1, '2026-04-05', 'Success', 'UPI', 20000.00),
456     (308, 3, 4, 2, '2026-04-15', 'Pending', 'Debit', 35000.00),
457     (309, 4, 5, 1, '2026-05-02', 'Success', 'Credit', 12000.00),
458     (310, 5, 1, 3, '2026-05-18', 'Pending', 'UPI', 70000.00);
459 -- Query: Count number of orders placed per month
460 • SELECT
461     YEAR(order_date) AS order_year,
462     MONTH(order_date) AS order_month,
463     MONTHNAME(order_date) AS order_month_name,
464     COUNT(*) AS total_orders
465 FROM Orders
466 GROUP BY YEAR(order_date), MONTH(order_date), MONTHNAME(order_date)
467 ORDER BY order_year, order_month;
```

Result Grid

	order_year	order_month	order_month_name	total_orders
▶	2026	1	January	2
	2026	2	February	2
	2026	3	March	27
	2026	4	April	3
	2026	5	May	2

i) Replace all 0 values in a column discount with NULL.

- used NULLIF to replace 0 with NULL to avoid showing unnecessary default values
- checked before and after to verify changes

04_null_handling*

```
489 -- Before placing NULL values
490 • select * from Products;
491 • UPDATE Products
492   SET discount = NULLIF(discount,0);
493
```

Result Grid

	product_id	name	description	price	category	discount
▶	1	Laptop	Intel i5, 8GB RAM, 512GB SSD	55000.00	Electronics	10.00
	2	Smartphone	6.5 inch display, 128GB storage	25000.00	Electronics	10.00
	3	Headphones	Wireless, Noise Cancelling	5000.00	Accessories	5.00
	4	Keyboard	Mechanical RGB keyboard	3000.00	Accessories	5.00
	5	Mouse	Wireless optical mouse	1500.00	Accessories	5.00
	6	Monitor	24 inch Full HD	10000.00	Electronics	10.00
	7	Backpack	Waterproof laptop backpack	2500.00	Bags	0.00
	8	Chair	Ergonomic office chair	6300.00	Furniture	15.00
	9	Table	Wooden study table	5400.00	Furniture	15.00
	10	Shoes	Running shoes, size 9	4000.00	Footwear	0.00
	11	Watch	Analog wrist watch	3500.00	Accessories	5.00
	12	Camera	DSLR 24MP with lens	45000.00	Electronics	10.00
	13	Printer	Laser printer with WiFi	12000.00	Electronics	10.00
	14	Notebook	200 pages ruled notebook	200.00	Stationery	0.00
	15	Pen	Gel pen pack of 5	150.00	Stationery	0.00
	16	T-Shirt	Cotton round neck T-shirt	700.00	Clothing	0.00
	17	Jeans	Slim fit denim jeans	1500.00	Clothing	0.00
	18	Jacket	Winter jacket, waterproof	3500.00	Clothing	0.00
	19	Adapter	USB-C adapter	200.00	Electronics	10.00
	20	Desk Lamp	LED desk lamp	1080.00	Furniture	15.00
	21	Sofa	Comfortable 3-seater sofa	27000.00	Furniture	15.00
	22	Dining Table	Wooden dining table	18000.00	Furniture	15.00
	23	Chair	Plastic chair	1350.00	Furniture	15.00
	24	Bed	King size bed	36000.00	Furniture	15.00
	25	Desk	Office work desk	9000.00	Furniture	15.00

04_null_handling*

Limit to 50000 rows

492

493 • UPDATE Products

494 SET discount = NULLIF(discount,0);

495

496 -- After replacing 0 with null values

497 • select * from Products;

498

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content:

	product_id	name	description	price	category	discount
▶	1	Laptop	Intel i5, 8GB RAM, 512GB SSD	55000.00	Electronics	10.00
	2	Smartphone	6.5 inch display, 128GB storage	25000.00	Electronics	10.00
	3	Headphones	Wireless, Noise Cancelling	5000.00	Accessories	5.00
	4	Keyboard	Mechanical RGB keyboard	3000.00	Accessories	5.00
	5	Mouse	Wireless optical mouse	1500.00	Accessories	5.00
	6	Monitor	24 inch Full HD	10000.00	Electronics	10.00
	7	Backpack	Waterproof laptop backpack	2500.00	Bags	NULL
	8	Chair	Ergonomic office chair	6300.00	Furniture	15.00
	9	Table	Wooden study table	5400.00	Furniture	15.00
	10	Shoes	Running shoes, size 9	4000.00	Footwear	NULL
	11	Watch	Analog wrist watch	3500.00	Accessories	5.00
	12	Camera	DSLR 24MP with lens	45000.00	Electronics	10.00
	13	Printer	Laser printer with WiFi	12000.00	Electronics	10.00
	14	Notebook	200 pages ruled notebook	200.00	Stationery	NULL
	15	Pen	Gel pen pack of 5	150.00	Stationery	NULL
	16	T-Shirt	Cotton round neck T-shirt	700.00	Clothing	NULL
	17	Jeans	Slim fit denim jeans	1500.00	Clothing	NULL
	18	Jacket	Winter jacket, waterproof	3500.00	Clothing	NULL
	19	Adapter	USB-C adapter	200.00	Electronics	10.00
	20	Desk Lamp	LED desk lamp	1080.00	Furniture	15.00
	21	Sofa	Comfortable 3-seater sofa	27000.00	Furniture	15.00
	22	Dining Table	Wooden dining table	18000.00	Furniture	15.00
	23	Chair	Plastic chair	1350.00	Furniture	15.00
	24	Bed	King size bed	36000.00	Furniture	15.00

j) Divide total_amount by quantity but avoid division by zero using NULLIF.

- added test case with quantity = 0
- used NULLIF to avoid division by zero error
- this returns NULL

04_null_handling* x

Limit to 50000 rows

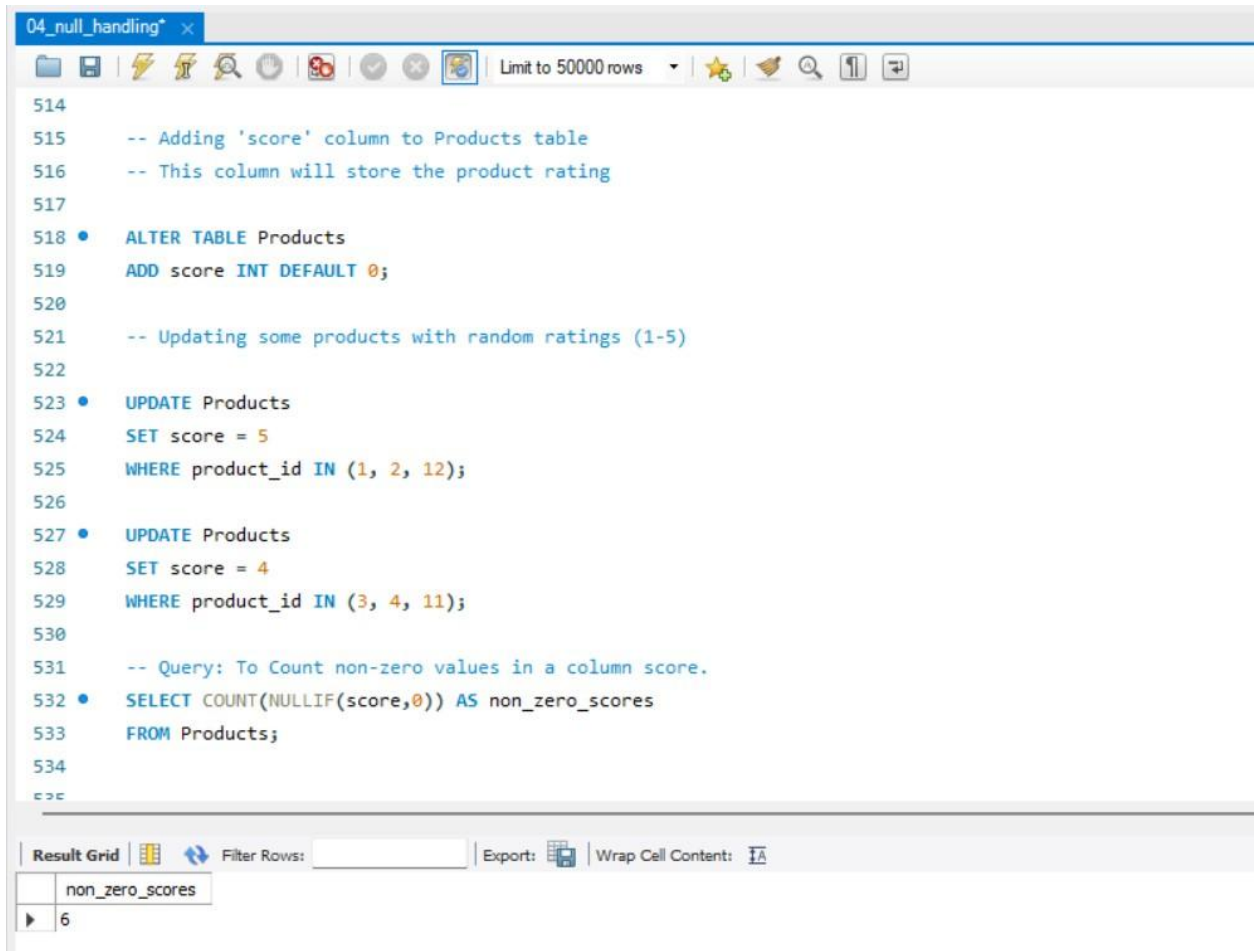
```
500 -- Inserting test data for Orders to demonstrate price_per_unit calculation
501 • INSERT INTO Orders (order_id, customer_id, product_id, quantity, order_date, status, payment_method, total_amount)
502 VALUES
503 (105, 5, 5, 0, '2026-04-05', 'Pending', 'COD', 3000.00); -- Zero quantity, should show NULL
504
505 -- Query : To Divide total_amount by quantity but avoid division by zero using NULLIF.
506 • SELECT
507     order_id,
508     customer_id,
509     product_id,
510     total_amount,
511     quantity,
512     total_amount / NULLIF(quantity, 0) AS price_per_unit
513 FROM Orders;
514
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: I

	order_id	customer_id	product_id	total_amount	quantity	price_per_unit
	22	1	1	55000.00	1	55000.000000
	24	24	24	450.00	3	150.000000
	25	25	25	80.00	1	80.000000
	26	26	23	600.00	1	600.000000
	105	5	5	3000.00	0	NULL
	301	1	1	15000.00	1	15000.000000
	302	2	2	22000.00	2	11000.000000
	303	3	3	12000.00	1	12000.000000
	304	4	4	45000.00	3	15000.000000
	305	5	5	9000.00	1	9000.000000
	306	1	2	55000.00	2	27500.000000
	307	2	3	20000.00	1	20000.000000
	308	3	4	35000.00	2	17500.000000

k) Count non-zero values in a column score.

- I added score column to store product ratings
- updated few records to have non-zero values
- used NULLIF to ignore 0 values while counting



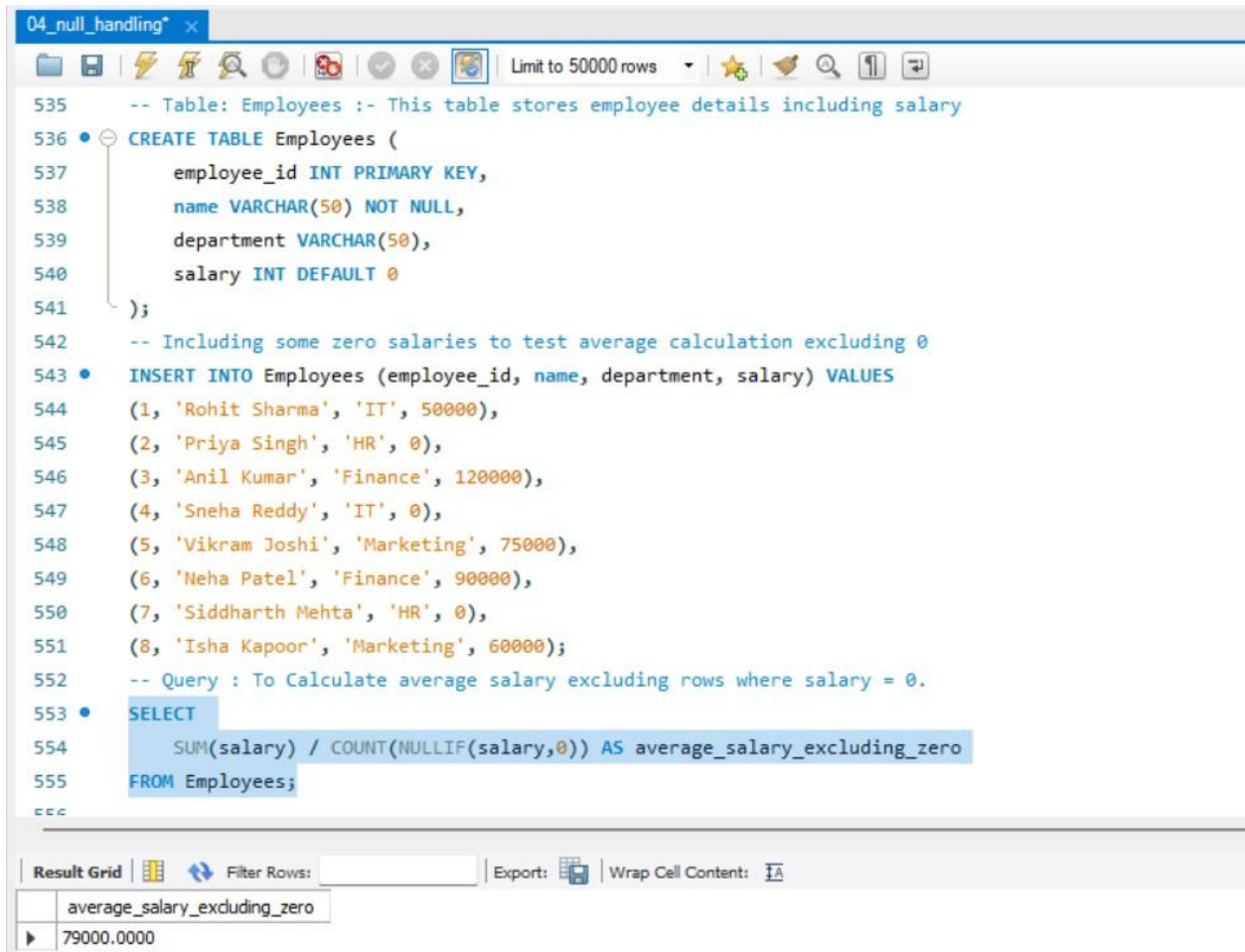
```
514
515 -- Adding 'score' column to Products table
516 -- This column will store the product rating
517
518 • ALTER TABLE Products
519   ADD score INT DEFAULT 0;
520
521 -- Updating some products with random ratings (1-5)
522
523 • UPDATE Products
524   SET score = 5
525   WHERE product_id IN (1, 2, 12);
526
527 • UPDATE Products
528   SET score = 4
529   WHERE product_id IN (3, 4, 11);
530
531 -- Query: To Count non-zero values in a column score.
532 • SELECT COUNT(NULLIF(score,0)) AS non_zero_scores
533   FROM Products;
534
535
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

non_zero_scores
6

I) Calculate average salary excluding rows where salary = 0.

- added some zero values samples
- used NULLIF to convert 0 into NULL so it doesn't affect calculation
- showed both SUM/COUNT and AVG approaches to compare methods



```
04_null_handling* x
-- Table: Employees :- This table stores employee details including salary
CREATE TABLE Employees (
    employee_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    department VARCHAR(50),
    salary INT DEFAULT 0
);
-- Including some zero salaries to test average calculation excluding 0
INSERT INTO Employees (employee_id, name, department, salary) VALUES
(1, 'Rohit Sharma', 'IT', 50000),
(2, 'Priya Singh', 'HR', 0),
(3, 'Anil Kumar', 'Finance', 120000),
(4, 'Sneha Reddy', 'IT', 0),
(5, 'Vikram Joshi', 'Marketing', 75000),
(6, 'Neha Patel', 'Finance', 90000),
(7, 'Siddharth Mehta', 'HR', 0),
(8, 'Isha Kapoor', 'Marketing', 60000);
-- Query : To Calculate average salary excluding rows where salary = 0.
SELECT
    SUM(salary) / COUNT(NULLIF(salary,0)) AS average_salary_excluding_zero
FROM Employees;
```

average_salary_excluding_zero
79000.0000

- Alternative Approach using AVG

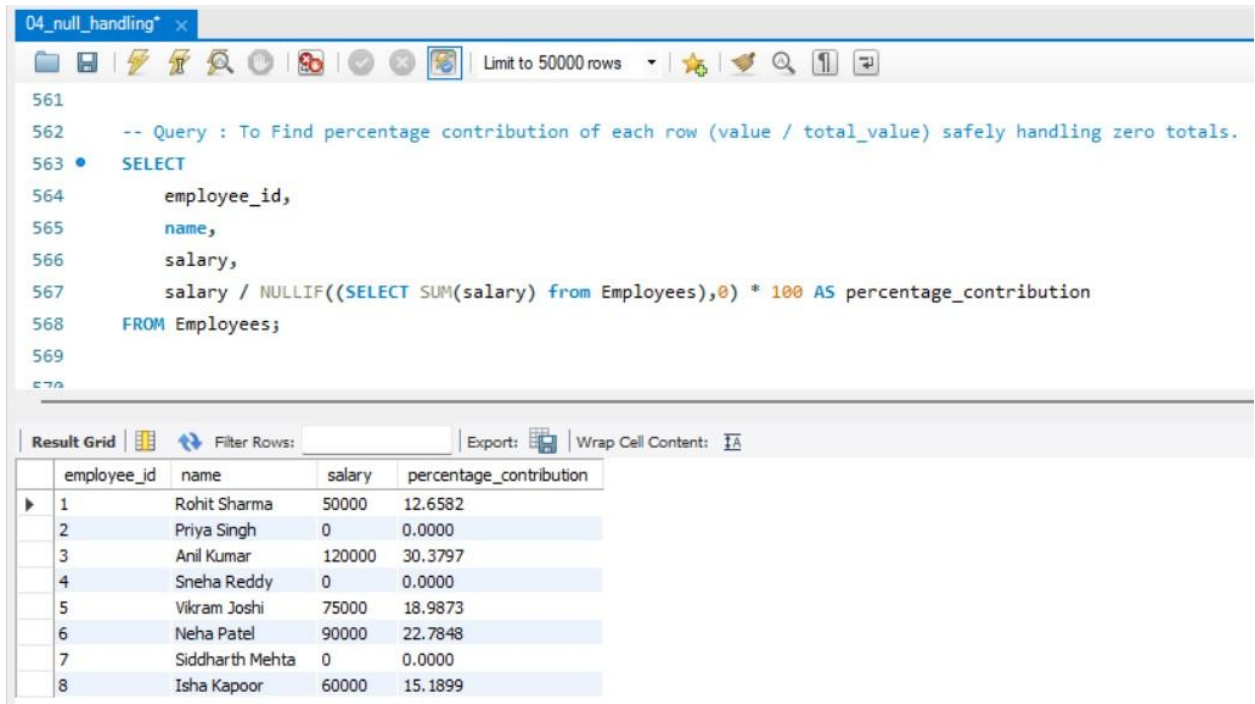


```
-- Alternative approach
SELECT AVG(NULLIF(salary,0)) as average_salary_excluding_zero FROM Employees;
```

average_salary_excluding_zero
79000.0000

m) Find percentage contribution of each row (value / total_value) safely handling zero totals.

- I wanted to find how much each employee contributes to total salary
- used subquery to get total salary of all employees
- used NULLIF to avoid division by zero in case total becomes 0
- calculated percentage



The screenshot shows a SQL IDE window titled "04_null_handling* x". The query editor contains the following SQL code:

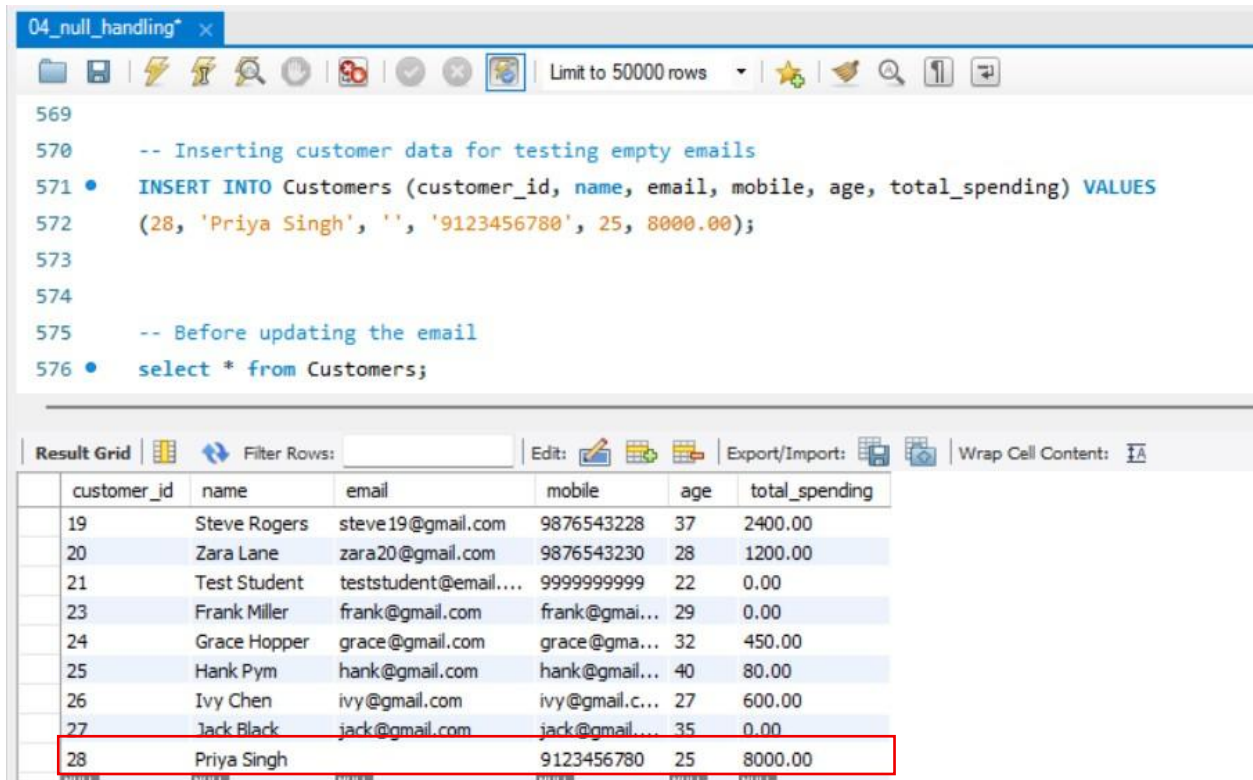
```
561
562 -- Query : To Find percentage contribution of each row (value / total_value) safely handling zero totals.
563 • SELECT
564     employee_id,
565     name,
566     salary,
567     salary / NULLIF((SELECT SUM(salary) from Employees),0) * 100 AS percentage_contribution
568 FROM Employees;
569
570
```

Below the query editor, the "Result Grid" is displayed, showing the results of the query. The grid has columns for employee_id, name, salary, and percentage_contribution. The results are as follows:

employee_id	name	salary	percentage_contribution
1	Rohit Sharma	50000	12.6582
2	Priya Singh	0	0.0000
3	Anil Kumar	120000	30.3797
4	Sneha Reddy	0	0.0000
5	Vikram Joshi	75000	18.9873
6	Neha Patel	90000	22.7848
7	Siddharth Mehta	0	0.0000
8	Isha Kapoor	60000	15.1899

n) Convert empty strings (") in a column email to NULL.

- inserted test data with empty string to check this case
- used NULLIF to convert empty string into NULL
- first screenshot shows email as empty string



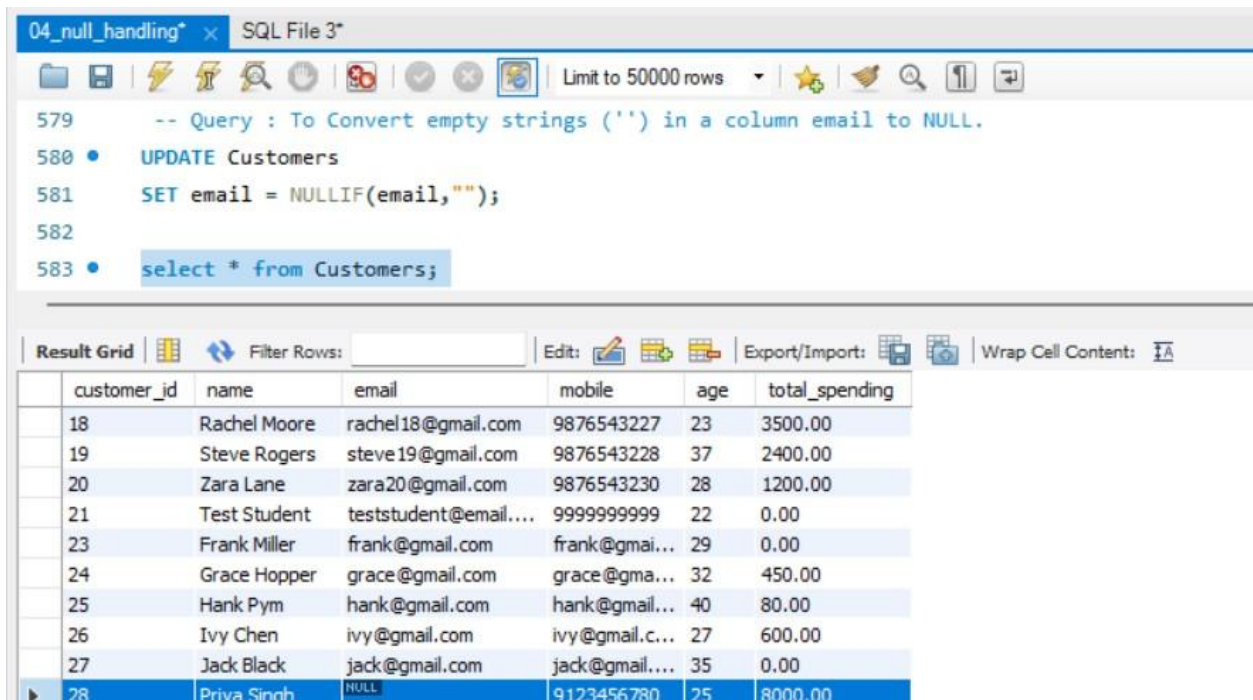
The screenshot shows a SQL Developer window titled "04_null_handling* x". The SQL editor contains the following code:

```
569
570 -- Inserting customer data for testing empty emails
571 • INSERT INTO Customers (customer_id, name, email, mobile, age, total_spending) VALUES
572 (28, 'Priya Singh', '', '9123456780', 25, 8000.00);
573
574
575 -- Before updating the email
576 • select * from Customers;
```

Below the code is the "Result Grid" showing the output of the query. The grid has columns: customer_id, name, email, mobile, age, total_spending. Row 28 is highlighted with a red box, showing the inserted data.

customer_id	name	email	mobile	age	total_spending
19	Steve Rogers	steve19@gmail.com	9876543228	37	2400.00
20	Zara Lane	zara20@gmail.com	9876543230	28	1200.00
21	Test Student	teststudent@email....	9999999999	22	0.00
23	Frank Miller	frank@gmail.com	frank@gmai...	29	0.00
24	Grace Hopper	grace@gmail.com	grace@gma...	32	450.00
25	Hank Pym	hank@gmail.com	hank@gmail...	40	80.00
26	Ivy Chen	ivy@gmail.com	ivy@gmail.c...	27	600.00
27	Jack Black	jack@gmail.com	jack@gmail....	35	0.00
28	Priya Singh		9123456780	25	8000.00

- second screenshot confirms the value is updated correctly



The screenshot shows a SQL Developer window titled "04_null_handling* x" with a sub-tab "SQL File 3*". The SQL editor contains the following code:

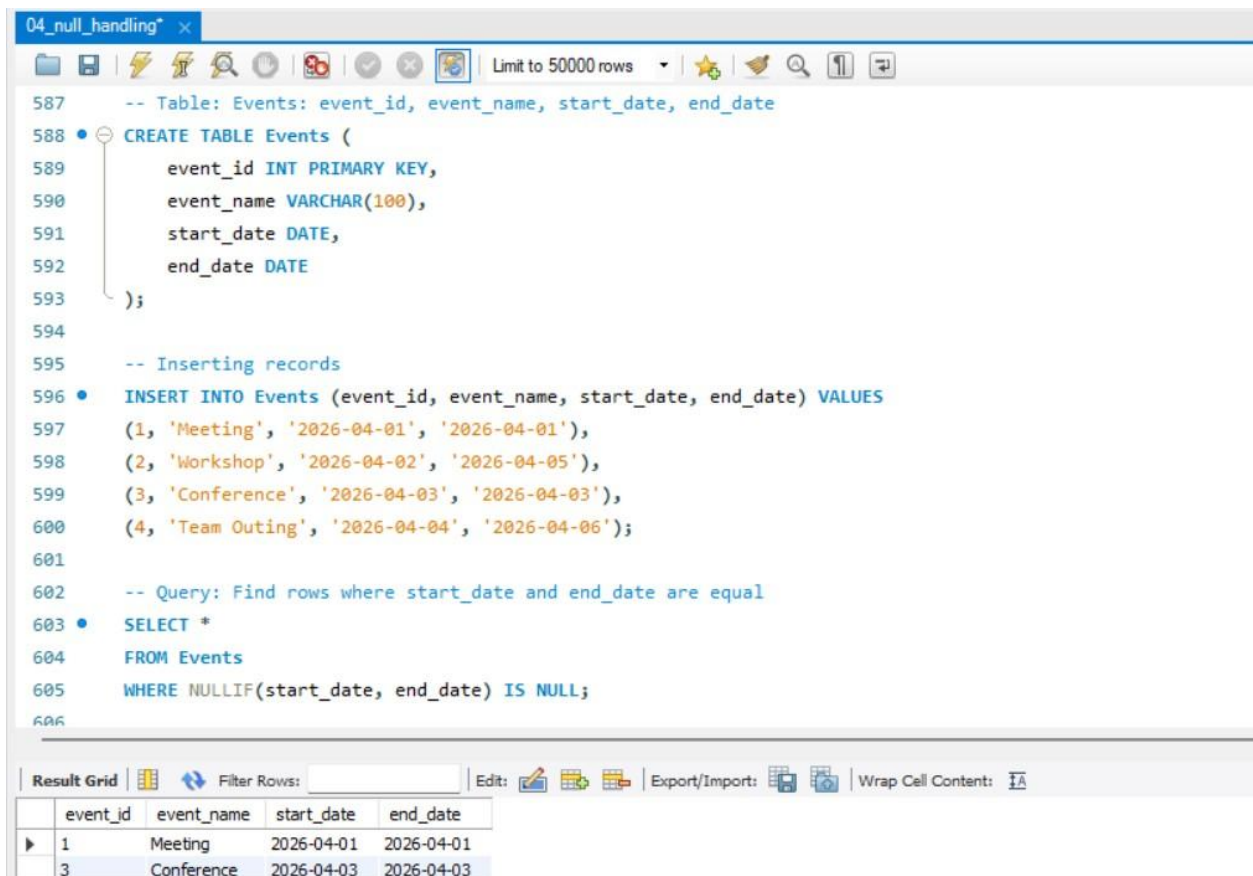
```
579 -- Query : To Convert empty strings (') in a column email to NULL.
580 • UPDATE Customers
581 SET email = NULLIF(email, '');
582
583 • select * from Customers;
```

Below the code is the "Result Grid" showing the output of the query. The grid has columns: customer_id, name, email, mobile, age, total_spending. Row 28 is highlighted, showing the email field updated to NULL.

customer_id	name	email	mobile	age	total_spending
18	Rachel Moore	rachel18@gmail.com	9876543227	23	3500.00
19	Steve Rogers	steve19@gmail.com	9876543228	37	2400.00
20	Zara Lane	zara20@gmail.com	9876543230	28	1200.00
21	Test Student	teststudent@email....	9999999999	22	0.00
23	Frank Miller	frank@gmail.com	frank@gmai...	29	0.00
24	Grace Hopper	grace@gmail.com	grace@gma...	32	450.00
25	Hank Pym	hank@gmail.com	hank@gmail...	40	80.00
26	Ivy Chen	ivy@gmail.com	ivy@gmail.c...	27	600.00
27	Jack Black	jack@gmail.com	jack@gmail....	35	0.00
28	Priya Singh	NULL	9123456780	25	8000.00

o) Identify rows where two columns start_date and end_date are equal using NULLIF.

- I wanted to find events that start and end on same day
- added sample data with both same and different dates
- used NULLIF to check if both dates are equal
- if equal, it returns NULL so filtered using IS NULL



The screenshot shows a SQL IDE window titled "04_null_handling". The script contains the following SQL statements:

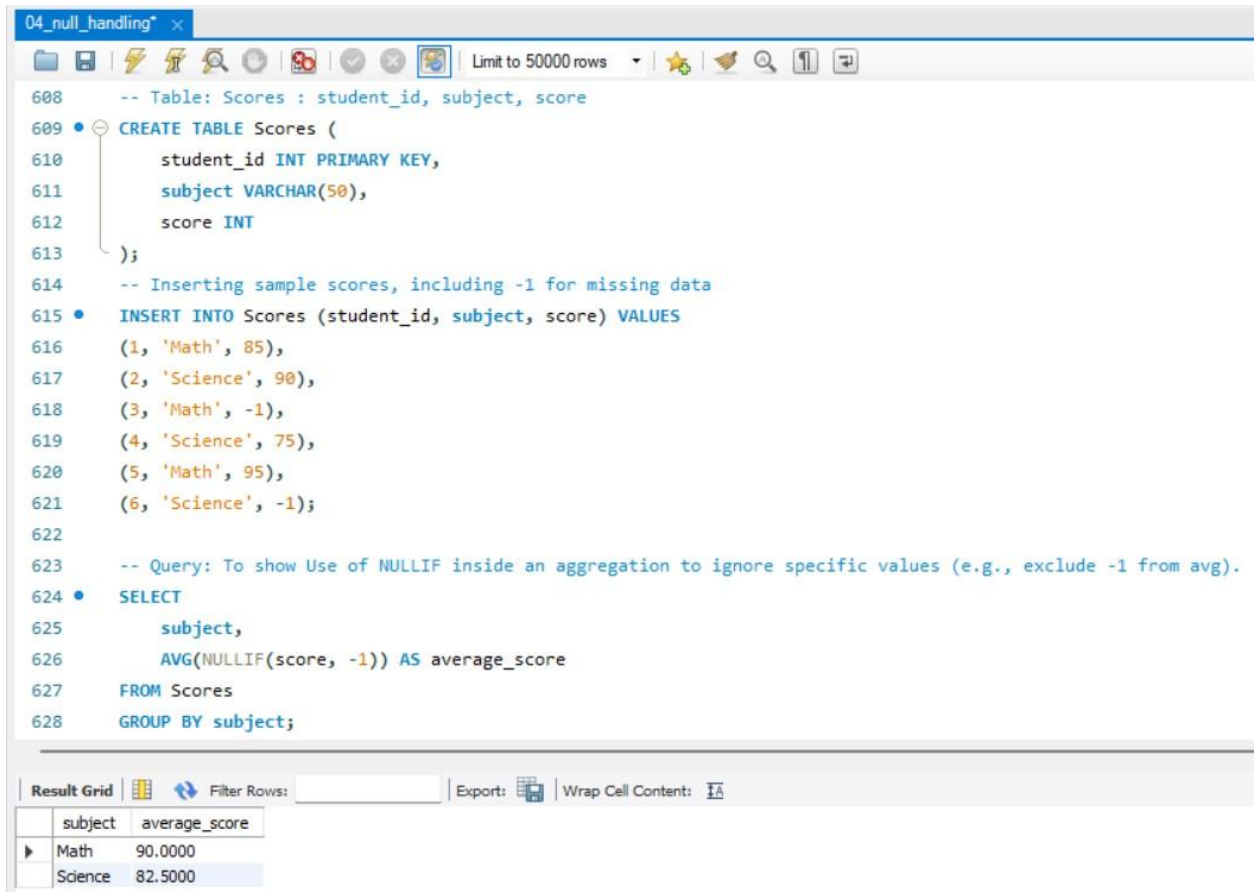
```
587 -- Table: Events: event_id, event_name, start_date, end_date
588 CREATE TABLE Events (
589     event_id INT PRIMARY KEY,
590     event_name VARCHAR(100),
591     start_date DATE,
592     end_date DATE
593 );
594
595 -- Inserting records
596 INSERT INTO Events (event_id, event_name, start_date, end_date) VALUES
597 (1, 'Meeting', '2026-04-01', '2026-04-01'),
598 (2, 'Workshop', '2026-04-02', '2026-04-05'),
599 (3, 'Conference', '2026-04-03', '2026-04-03'),
600 (4, 'Team Outing', '2026-04-04', '2026-04-06');
601
602 -- Query: Find rows where start_date and end_date are equal
603 SELECT *
604 FROM Events
605 WHERE NULLIF(start_date, end_date) IS NULL;
```

The "Result Grid" at the bottom shows the following data:

	event_id	event_name	start_date	end_date
▶	1	Meeting	2026-04-01	2026-04-01
	3	Conference	2026-04-03	2026-04-03

p) Use NULLIF inside an aggregation to ignore specific values (e.g., exclude -1 from avg).

- I wanted to calculate average score but ignore invalid values (-1)
- added -1 as missing data to test this case
- used NULLIF to convert -1 into NULL so AVG ignores it
- grouped by subject to get average per subject



The screenshot shows a SQL IDE window titled "04_null_handling". The script contains the following SQL code:

```
608 -- Table: Scores : student_id, subject, score
609 CREATE TABLE Scores (
610     student_id INT PRIMARY KEY,
611     subject VARCHAR(50),
612     score INT
613 );
614 -- Inserting sample scores, including -1 for missing data
615 INSERT INTO Scores (student_id, subject, score) VALUES
616 (1, 'Math', 85),
617 (2, 'Science', 90),
618 (3, 'Math', -1),
619 (4, 'Science', 75),
620 (5, 'Math', 95),
621 (6, 'Science', -1);
622
623 -- Query: To show Use of NULLIF inside an aggregation to ignore specific values (e.g., exclude -1 from avg).
624 SELECT
625     subject,
626     AVG(NULLIF(score, -1)) AS average_score
627 FROM Scores
628 GROUP BY subject;
```

Below the script, the "Result Grid" shows the output of the query:

	subject	average_score
▶	Math	90.0000
	Science	82.5000

q) Replace a value only when two columns match (hint: combine NULLIF + CASE).

- I wanted to update status only for events happening on same day
- used NULLIF to check if start_date and end_date are same
- used CASE to update only matching rows
- first screenshot shows initial status values

04_null_handling* x

Limit to 50000 rows

```
629
630 • ALTER TABLE Events
631   ADD status VARCHAR(50) DEFAULT 'Pending';
632
633   -- Before replacing
634 • SELECT * from Events;
635
```

Result Grid

	event_id	event_name	start_date	end_date	status
▶	1	Meeting	2026-04-01	2026-04-01	Pending
	2	Workshop	2026-04-02	2026-04-05	Pending
	3	Conference	2026-04-03	2026-04-03	Pending
	4	Team Outing	2026-04-04	2026-04-06	Pending
*	NULL	NULL	NULL	NULL	NULL

- second screenshot confirms only those rows are updated

04_null_handling* x

Limit to 50000 rows

```
637   -- Query : To Replace a value only when two columns match (hint: combine NULLIF + CASE).
638 • UPDATE Events
639   SET status = CASE
640     WHEN NULLIF(start_date, end_date) IS NULL THEN 'Same Day Event'
641     ELSE status
642   END;
643
644   -- After replacing
645 • SELECT * from Events;
```

Result Grid

	event_id	event_name	start_date	end_date	status
▶	1	Meeting	2026-04-01	2026-04-01	Same Day Event
	2	Workshop	2026-04-02	2026-04-05	Pending
	3	Conference	2026-04-03	2026-04-03	Same Day Event
	4	Team Outing	2026-04-04	2026-04-06	Pending
*	NULL	NULL	NULL	NULL	NULL